

ARQUITECTURA DE SOFTWARE FORMALMENTE VERIFICADO

Verificador de Coherencia ABAC

Catálogo sistemático de **Tipos de Datos, Estructuras Compuestas e Invariantes Formales** en el paquete @grupo1/abac-verifier.

MOTOR CENTRAL

Dafny 4 (SMT / Z3)

CAPA DE FRONTERA

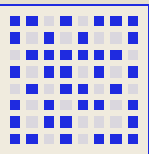
TypeScript Strict FFI

GARANTÍA CENTRAL

Invariante Inductivo

INTEGRANTES

GRUPO 1



01 Jara Mamani, Mariel Alisson

02 Mestas Zegarra, Christian Raúl

03 Noa Camino, Yenaro Joel

04 Sequeiros Condori, Luis Gustavo

Tipos de Datos Primitivos y Base

Para garantizar **decidibilidad automática en SMT (Z3)**, el sistema restringe los atributos a tipos escalares finitos y datatypes algebraicos en Dafny:

01

BoundedInt (Enteros)

Entero acotado a un intervalo cerrado finito $[l_o, h_i]$. Evita la no-terminación y mantiene la aritmética lineal en dominios seguros.

```
// Ejemplo en el esquema v1:  
clearance: BoundedInt (1..5)  
hour: BoundedInt (0..23)  
sensitivity: BoundedInt (1..5)
```

02

Enum (Enumeraciones)

Conjunto finito y explícito de cadenas de texto (seq<string>) con cota estricta $0 < |valores| \leq 16$.

```
// Ejemplo en el esquema v1:  
role ∈ {medic, nurse, admin}  
dept ∈ {cardio, radio, er}  
action ∈ {read, write}
```

03

Bool y Datatypes Base

Booleanos puros y uniones discriminadas (datatypes) que modelan efectos de autorización y valores universales.

```
// Ejemplo en Dafny:  
emergency: Bool (true | false)  
datatype Effect = Permit | Deny  
datatype Opt<T> = None | Some(v)
```

Tipos de Datos Compuestos

Estructuras algebraicas recursivas y colecciones que representan el **AST de predicados**, el **conjunto de reglas** y las **peticiones**:

Pred (AST de Predicados)

Datatype recursivo inductivo que construye árboles lógicos con átomos de comparación, negación, conjunción y disyunción.

```
datatype Pred =  
  | Atom(family, attr, op, lit, num)  
  | Not(p: Pred)  
  | And(a: Pred, b: Pred)  
  | Or(a: Pred, b: Pred)
```

Rule y Ruleset

Regla compuesta por un identificador numérico, su efecto y su predicado objetivo; un Ruleset es una secuencia seq<Rule>.

```
datatype Rule =  
  Rule(id: int, effect: Effect,  
    target: Pred)  
  
type Ruleset = seq<Rule>  
// Ej: Rule(1, Permit, role ==  
  "medic")
```

AttrDecl y Schema

Contrato de atributos PIP. Cada declaración fija familia, nombre, tipo y dominio; el esquema es la secuencia seq<AttrDecl>.

```
datatype AttrDecl = AttrDecl(  
  family: string, name: string,  
  attrType: AttrType,  
  enumVals: seq<string>,  
  lo: int, hi: int)  
type Schema = seq<AttrDecl>
```

Request, Witness y AddResult

Petición posicional seq<Value>, testigos de contraejemplo para fallos de coherencia y tipos suma para resultados seguros.

```
type Request = seq<Value>  
datatype Witness = Witness(  
  req, ruleIds, failedProperty, expl)  
datatype AddResult =  
  Accepted(Ruleset) |  
  Rejected(Witness)
```

Invariantes Estructurales y Contratos

Predicados de validez (**invariantes**) y cláusulas formales de contrato (**requires** / **ensures**) implementados en Dafny:

INVARIANTE / PREDICADO	CONDICIÓN FORMAL EN CÓDIGO	OBJETIVO Y GARANTÍA	UBICACIÓN
predicate SchemaWellFormed	$\forall d \in \text{schema} \cdot \text{ValidDecl}(d) \wedge \forall i < j \cdot \text{DeclKey}(s[i]) \neq \text{DeclKey}(s[j])$	Unicidad de atributos por familia (<code>DeclKey</code>), no vacíos y cotas coherentes <code>lo ≤ hi</code> .	<code>Schema.dfy</code> (1.41)
predicate ValidRule & ValidAtom	<code>LookupDecl(schema, a.attr).Some? $\wedge$ ValidOp(a.op) $\wedge$ a.lit $\in$ enumVals</code>	Tipado estricto: toda regla usa atributos declarados con operadores y literales válidos (prohíbe <code>role > 3</code>).	<code>Syntax.dfy</code> (1.37, 54)
predicate ValidRequest	<code> req == schema $\wedge$ $\forall i \cdot \text{ValueMatchesDecl}(req[i], schema[i])$</code>	Petición total por construcción; peticiones fuera de dominio se rechazan en la frontera TypeScript.	<code>Schema.dfy</code> (1.52)

PRECONDICIÓN (REQUIRES)

requires **SchemaWellFormed(schema)**

Exige atributos y tipos válidos antes de enumerar o evaluar peticiones.

PRECONDICIÓN (REQUIRES)

requires **ValidRequest(req, schema)**

Exige que la petición sea completa y pertenezca al dominio declarado.

POSTCONDICIÓN (ENSURES)

ensures **Coherent(RemoveById(R, id), schema)**

Garantiza formalmente que retirar una regla siempre preserva la coherencia.

CONTRATOS FORMALES: Toda función y lema declara sus hipótesis (requires) y garantías verificadas (ensures).

Invariantes de Coherencia y Transición

El verificador formaliza la conjunción $\text{Coherent}(R, \text{schema}) \equiv P1 \wedge P2 \wedge P3 \wedge P4$ y demuestra su preservación inductiva:

P1

P1 • Consistencia (No Contradicción)

Ninguna petición satisface simultáneamente una regla Permit y una Deny: $\neg(\text{PermitApplies} \wedge \text{DenyApplies})$.

P2

P2 • Satisfacibilidad de Reglas

Toda regla r debe activarse en al menos una petición: $\exists \text{ req} \cdot \text{Applies}(\text{req}, r)$. Prohíbe reglas vacías.

P3

P3 • No Sombreamiento (No Subsunción)

Cada regla decide en solitario al menos una petición. Ninguna regla está anulada o enmascarada por otra existente.

P4

P4 • Validez Tipográfica de Esquema

Todas las reglas cumplen $\text{AllRulesValid}(\text{ruleset}, \text{schema})$ tipadas estrictamente contra el PIP.

INVARIANTE INDUCTIVO DE ESTADO

Todo ruleset alcanzable mediante la API protegida es coherente por construcción.

TryAdd(R, r) → Accepted(R') | Rejected(witness)

TryUpdate(R, id, r) → Accepted(R') | Rejected(witness)

TryRemove(R, id) → R' (Preserva coherencia)

✓ EmptyCoherent: Ruleset vacío es vacuamente coherente.

✓ AddPreserves: Guarded Add rechaza violaciones con contraejemplo.

✓ RemovePreservesCoherent: Probado por monotonía de subconjuntos.

VERIFICACIÓN FORMAL DE REGLAS DE ACCESO —

¡Muchas Gracias!

Verificación formal de coherencia para políticas ABAC.
Dafny decide formalmente; TypeScript envuelve con seguridad.

Paquete: @grupo1/abac-verifier

Garantía: P1 · P2 · P3 · P4 Coherent

Integrantes: Jara · Mestas · Noa · Sequeiros